
北京交通大學

BEIJING JIAOTONG UNIVERSITY

# 《汇编与接口技术》
# 研究型实验报告

| | |
|---|---|
| 实验名称： | 图像灰度转换的 NEON 优化 |
| 学　号： | 24281068 |
| 姓　名： | 刘思诚 |
| 学　院： | 计算机科学与技术学院 |
| 日　期： | 2026 年 4 月 20 日 |

# 一、实验目的

在鲲鹏 920 (ARMv8-A) 平台上，使用 NEON SIMD 指令集对 RGB→Gray 灰度转换进行优化，通过对比纯 C、NEON Intrinsics、NEON 汇编、多路循环展开等方案，探究 SIMD 并行加速的实际效果、最优展开度以及性能瓶颈所在。

# 二、算法原理

## 2.1 灰度转换公式

RGB 转灰度公式：

$$\text{Gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

由于浮点运算速度慢，在本次实验中使用整数近似（系数乘以 $2^8 = 256$ 后用移位还原）：

$$\text{Gray} = (77 \cdot R + 150 \cdot G + 29 \cdot B) \gg 8$$

## 2.2 并行性分析

每个像素的三个通道（R/G/B）独立计算，像素之间无数据依赖。输入为 8bit 三通道交错排列（RGBRGBRGB...），输出为 8bit 单通道。这种"相同操作作用于大量数据"的模式天然适合 SIMD 并行处理。ARMv8 NEON 支持 128bit 向量寄存器，一次可处理 8 个 16bit 元素或 16 个 8bit 元素。

# 三、实验环境

| 项目 | 参数 |
| --- | --- |
| 处理器 | 鲲鹏 920 (ARMv8, implementer=0x48, part=0xd01) |
| 架构 | AArch64, 支持 NEON (asimd) |
| 测试数据 | 随机 RGB 数据 (1920×1080) + 真实图片 (1624×1080) |
| 计时方法 | clock_gettime(CLOCK_MONOTONIC), 纳秒精度 |

# 四、实验方案

## 4.1 五种实现

| 版本 | 描述 | 每轮处理 |
| --- | --- | --- |
| C (baseline) | 纯 C 逐像素标量循环 | 1 像素 |
| NEON x1 | 基础 NEON，无展开 | 8 像素 |
| NEON x2 | 2 路循环展开 | 16 像素 |
| NEON x4 | 4 路循环展开 | 32 像素 |
| NEON x8 | 8 路循环展开 | 64 像素 |
| NEON x16 | 16 路循环展开 | 128 像素 |

## 4.2 关键 NEON 指令

vld_u8：一次加载 3*8 字节，自动将 RGB 分为三个通道

vmovl_u8：将 8 位扩展至 16 位，防止乘法溢出

vmulq_u16：16 位乘法

vmlaq_u16：16 位向量乘加

vshrn_n_u16：将通过移位将 16 位变为 8 位

vstl_u8：存储 8 位的灰度计算结果

## 4.3 编译指令

gcc -O2 -march=armv8-a+simd -Wall -o gray_test \ main.c gray_c.c gray_neon.c gray_neon_asm.S \gray_neon_x2.c gray_neon_x4.c gray_neon_x8.c gray_neon_x16.c -lm

# 五、实验结果

## 5.1 随机数据测试 (1920×1080 = 2,073,600 像素)

| 版本 | 耗时 (ms) | 加速比 |
| --- | --- | --- |
| C | 4.17 | 1.00× |
| NEON(x1) | 2.68 | 1.56× |
| NEON(Asm) | 2.67 | 1.56× |

| 版本 | 耗时 (ms) | 加速比 |
| --- | --- | --- |
| NEON(x4) | 2.18 | 1.91× |

## 5.3 真实图片测试 (1624×1080 = 1,753,920 像素)

### 5.3.1 真实图片输入输出

图 1 test_input.png

图 2 test_output.png

### 5.3.2 真实图图片测试运行速度对比

```
[stu010@localhost gray_image_test]$ ./gray_image_test test_input1.png
Loaded 'test_input1.png': 1276x958, ch=3 (forced RGB)

Correctness:
  NEON(x1): OK
  NEON(x2): OK
  NEON(x4): OK
  NEON(x8): OK
  NEON(x16):OK

Benchmark (1222408 pixels):
  C:              2.03 ms   (1.00x)
  NEON(x1) :       1.20 ms   (1.70x)
  NEON(x2) :       1.03 ms   (1.97x)
  NEON(x4) :       0.89 ms   (2.28x)
  NEON(x8) :       1.18 ms   (1.72x)
  NEON(x16):       1.16 ms   (1.74x)

Saved grayscale: 'gray_output.png'
```

图 3 日志输出

| 版本 | 耗时 (ms) | 加速比 |
| --- | --- | --- |
| C | 2.93 | 1.00× |
| NEON(x1) | 1.71 | 1.71× |
| NEON(x2) | 1.50 | 1.95× |

| 版本 | 耗时 (ms) | 加速比 |
| --- | --- | --- |
| NEON(x4) | 1.36 | 2.15× |
| NEON(x8) | 1.72 | 1.70× |
| NEON(x16) | 1.70 | 1.72× |

# 六、结果分析

## 6.1 理论加速比与实际加速比的差距

NEON 一次处理 8 个像素（8 路 SIMD lane），理论上可达 8× 加速。实际最优仅 2.15×，差距来自以下原因。

## 6.2 展开度 x8/x16 性能反降分析

核心原因：NEON 寄存器溢出。

AArch64 NEON 共有 32 个 128bit 向量寄存器 (v0-v31)：

| 展开度 | 最少寄存器需求 | 寄存器是否溢出 |
| --- | --- | --- |
| x1 | 7 (3 数据 + 3 系数 + 1 结果) | 否 |
| x4 | 19 (12 数据 + 3 + 4) | 否 |
| x8 | 35 (24 数据 + 3 + 8) > 32 | 溢出 |
| x16 | 67 (48 + 3 + 16) | 严重溢出 |

编译器被迫将溢出部分临时存入栈内存（spill/fill），额外的访存操作消耗了大量时间，导致展开收益被完全抵消甚至反转。这也是 x8（1.72 ms）几乎退化到 x1（1.71 ms）水平的原因。

## 6.3 随机数据与真实图片的性能差异

真实图片（2.93 ms）比随机数据（4.17 ms）快约 42%——真实图像像素具有空间局部性，相邻像素颜色相似，CPU 硬件预取器效率更高，Cache 命中率更高。

# 七、总结

## 7.1 重难点分析

1. RGB 转灰度的 NEON 向量化实现。将标量公式 Gray = (77R + 150G + 29B) >> 8 映射为 NEON 指令序列是本次实验的核心。关键在于理解 vld3 加载解交织 → vmovl 扩位 → vmulq/vmlaq 乘累加 → vshrn 窄化右移 → vst1 存储这一完整流水线，以及每个阶段数据类型（uint8x8_t → uint16x8_t → uint8x8_t）的变换。

2. ARM NEON 汇编的编写与调试。手写汇编需要对 AArch64 指令集、寄存器约定（x0-x2 传参、x0 返回值）、以及 NEON 寄存器文件（v0-v31）有清晰的理解。ld3/st1 等指令的排列修饰符（.8b、.8h）直接决定操作宽度，写错会导致数据错位或静默错误。此外，汇编代码缺乏类型检查，一旦 shrn 写错移位量或 uxtl 写错源/目标宽度，编译不会报错，只能通过逐字节比对验证才能暴露。

## 7.2 心得体会

在做这个实验之前，我对 SIMD 的理解停留在概念层面，知道它能加速但对其原理缺乏直观感受。通过亲手编写 NEON intrinsics 和手写汇编，才真正体会到 SIMD 的核心思想——一条指令同时操作多个数据，比如 vld3 一次性加载并解交织 8 个像素的 RGB 三个通道，而 C 语言中这需要循环逐个处理。手写汇编部分印象较深，在 C 语言中看似简单的 rgb[i*3] 地址计算，到汇编中需要手动实现 add x4, x3, x3, lsl #1 再加基址，这让人更清晰地认识到高级语言和编译器底层所隐藏的工作量。调试汇编时也遇到了一些问题，例如 shrn 移位量写错后编译不报错，只能靠 verify()逐字节比对才能发现输出偏差，这说明了在底层优化中，自动化验证手段是不可或缺的。从完全不了解 NEON 到能够独立完成向量化代码的编写与调优，这个过程加深了对 ARM 体系结构的理解，也为后续接触其他 SIMD 平台积累了可迁移的经验。

# 八、附录

## 8.1 原始终端日志

### 编译日志

```
[stu010@localhost gray_image_test]$  echo "=== Build ===" && make clean && make
=== Build ===
rm -f main.o gray_c.o gray_neon.o gray_neon_asm.o gray_neon_x2.o gray_neon_x4.o gray_neon_x8.o gray_neon_x16.o gray_image_test
gcc -O2 -march=armv8-a+simd -Wall -c main.c -o main.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_c.c -o gray_c.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_neon.c -o gray_neon.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_neon.S -o gray_neon_asm.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_neon_x2.c -o gray_neon_x2.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_neon_x4.c -o gray_neon_x4.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_neon_x8.c -o gray_neon_x8.o
gcc -O2 -march=armv8-a+simd -Wall -c gray_neon_x16.c -o gray_neon_x16.o
gcc -O2 -march=armv8-a+simd -Wall -o gray_image_test main.o gray_c.o gray_neon.o gray_neon_asm.o gray_neon_x2.o gray_neon_x4.o gray_neon_x8.o gray_neon_x16.o -lm
```

### 随机数据测试 (1920×1080)

```
[stu010@localhost gray_image_test]$ cd ~/gray_neon_advanced
[stu010@localhost gray_neon_advanced]$  echo "=== Random Data Test ===" && ./gray_test
=== Random Data Test ===
Testing RGB to Gray conversion on 1920x1080 image (2073600 pixels)

C:                4181701 ns  (    4.18 ms)
NEON(C):          2683542 ns  (    2.68 ms)
NEON(Asm):        2657963 ns  (    2.66 ms)
NEON(x4):         2222987 ns  (    2.22 ms)

OK NEON(C)  results match C baseline
OK NEON(Asm) results match C baseline
OK NEON(x4)  results match C baseline
```

# 真实图片测试 (1624×1080)

```
[stu010@localhost gray_image_test]$ echo "=== Real Image Test ===" && ./gray_image_test test_input.jpg gray_output.png
=== Real Image Test ===
Loaded 'test_input.jpg': 1624x1080, ch=3 (forced RGB)

Correctness:
  NEON(x1): OK
  NEON(x2): OK
  NEON(x4): OK
  NEON(x8): OK
  NEON(x16):OK

Benchmark (1753920 pixels):
  C:              2.93 ms   (1.00x)
  NEON(x1) :       1.71 ms   (1.71x)
  NEON(x2) :       1.50 ms   (1.95x)
  NEON(x4) :       1.36 ms   (2.15x)
  NEON(x8) :       1.72 ms   (1.70x)
  NEON(x16):       1.70 ms   (1.72x)

Saved grayscale: 'gray_output.png'
```

# 相关代码

## 1.baseline（c）

```c
#include <stdint.h>

void rgb2gray_c(unsigned char *rgb, unsigned char *gray, int num_pixels)
{
    for (int i = 0; i < num_pixels; i++) {
        int r = rgb[i * 3 + 0];
        int g = rgb[i * 3 + 1];
        int b = rgb[i * 3 + 2];
        // gray = 0.299*R + 0.587*G + 0.114*B
        int sum = 77 * r + 150 * g + 29 * b;
        gray[i] = sum / 256;
    }
}
```

## 2.一路展开汇编

```
.text
.global rgb2gray_neon_asm

rgb2gray_neon_asm:
    movi v4.8h,77
    movi v5.8h,150
    movi v6.8h,29
    sub x2,x2,8
    mov x3,0
.Lloop:
    cmp x3,x2
    bge .Ltail

    add x4,x3,x3,lsl 1
    add x4,x0,x4

    ld3 {v0.8b,v1.8b,v2.8b},[x4]

    uxtl v0.8h,v0.8b
    uxtl v1.8h,v1.8b
    uxtl v2.8h,v2.8b

    mul  v0.8h,v0.8h, v4.8h
    mla  v0.8h,v1.8h, v5.8h
    mla  v0.8h,v2.8h, v6.8h
    shrn v3.8b,v0.8h, 8

    add x4,x1,x3
    st1 {v3.8b},[x4]

    add x3,x3,8
    b .Lloop

.Ltail:
```

```
    add x2,x2,8
.Ltail_loop:
    cmp x3,x2
    bge .Ldone

    add x4,x3,x3,lsl 1
    add x4,x0,x4

    ldrb w5,[x4]
    ldrb w6,[x4, 1]
    ldrb w7,[x4, 2]

    mov w8,77
    mul w5,w5,w8
    mov w8,150
    madd w5,w6,w8,w5
    mov w8,29
    madd w5,w7,w8,w5
    lsr w5,w5,8

    strb w5,[x1,x3]

    add x3,x3,1
    b .Ltail_loop

.Ldone:
    ret
```

## 3.两路循环展开

```
#include <arm_neon.h>
#include <stdint.h>

/* 2 路循环展开：每次处理 16 像素 */
void rgb2gray_neon_x2(const uint8_t *rgb, uint8_t *gray, int n)
{
    uint16x8_t rc = vdupq_n_u16(77);
    uint16x8_t gc = vdupq_n_u16(150);
    uint16x8_t bc = vdupq_n_u16(29);
    int i;


    for (i = 0; i <= n - 16; i += 16) {
        const uint8_t *p = &rgb[i * 3];


        uint8x8x3_t d0 = vld3_u8(p + 0);
        uint8x8x3_t d1 = vld3_u8(p + 24);


        uint16x8_t s0 = vmulq_u16(vmovl_u8(d0.val[0]), rc);
        s0 = vmlaq_u16(s0, vmovl_u8(d0.val[1]), gc);
        s0 = vmlaq_u16(s0, vmovl_u8(d0.val[2]), bc);


        uint16x8_t s1 = vmulq_u16(vmovl_u8(d1.val[0]), rc);
        s1 = vmlaq_u16(s1, vmovl_u8(d1.val[1]), gc);
        s1 = vmlaq_u16(s1, vmovl_u8(d1.val[2]), bc);


        vst1_u8(&gray[i + 0], vshrn_n_u16(s0, 8));
        vst1_u8(&gray[i + 8], vshrn_n_u16(s1, 8));
    }


    for (; i < n; i++)
        gray[i] = (77 * rgb[i*3] + 150 * rgb[i*3+1] + 29 * rgb[i*3+2]) >> 8;
}
```

## 4.四路循环展开

```
#include <arm_neon.h>
#include <stdint.h>

void rgb2gray_neon_x4(const uint8_t *rgb, uint8_t *gray, int num_pixels)
{
    uint16x8_t rc = vdupq_n_u16(77);
    uint16x8_t gc = vdupq_n_u16(150);
    uint16x8_t bc = vdupq_n_u16(29);
    int i = 0;
```

```
    if (num_pixels >= 32) {
        for (i = 0; i + 31 < num_pixels; i += 32) {
            const uint8_t *p = &rgb[i * 3];

            uint8x8x3_t d0 = vld3_u8(p + 0);
            uint8x8x3_t d1 = vld3_u8(p + 24);
            uint8x8x3_t d2 = vld3_u8(p + 48);
            uint8x8x3_t d3 = vld3_u8(p + 72);

            uint16x8_t s0 = vmulq_u16(vmovl_u8(d0.val[0]), rc);
            s0 = vmlaq_u16(s0, vmovl_u8(d0.val[1]), gc);
            s0 = vmlaq_u16(s0, vmovl_u8(d0.val[2]), bc);

            uint16x8_t s1 = vmulq_u16(vmovl_u8(d1.val[0]), rc);
            s1 = vmlaq_u16(s1, vmovl_u8(d1.val[1]), gc);
            s1 = vmlaq_u16(s1, vmovl_u8(d1.val[2]), bc);

            uint16x8_t s2 = vmulq_u16(vmovl_u8(d2.val[0]), rc);
            s2 = vmlaq_u16(s2, vmovl_u8(d2.val[1]), gc);
            s2 = vmlaq_u16(s2, vmovl_u8(d2.val[2]), bc);

            uint16x8_t s3 = vmulq_u16(vmovl_u8(d3.val[0]), rc);
            s3 = vmlaq_u16(s3, vmovl_u8(d3.val[1]), gc);
            s3 = vmlaq_u16(s3, vmovl_u8(d3.val[2]), bc);

            vst1_u8(&gray[i + 0],  vshrn_n_u16(s0, 8));
            vst1_u8(&gray[i + 8],  vshrn_n_u16(s1, 8));
            vst1_u8(&gray[i + 16], vshrn_n_u16(s2, 8));
            vst1_u8(&gray[i + 24], vshrn_n_u16(s3, 8));
        }
    }

    for (; i < num_pixels; i++) {
        gray[i] = (77 * rgb[i*3] + 150 * rgb[i*3+1] + 29 * rgb[i*3+2]) / 256;
    }
}
```

## 5.八路循环展开

```
#include <arm_neon.h>
#include <stdint.h>

void rgb2gray_neon_x8(const uint8_t *rgb, uint8_t *gray, int n)
{
    uint16x8_t rc = vdupq_n_u16(77);
    uint16x8_t gc = vdupq_n_u16(150);
    uint16x8_t bc = vdupq_n_u16(29);
    int i;

    for (i = 0; i <= n - 64; i += 64) {
        const uint8_t *p = &rgb[i * 3];

        uint8x8x3_t d0 = vld3_u8(p);
        uint16x8_t s0 = vmulq_u16(vmovl_u8(d0.val[0]), rc);
        s0 = vmlaq_u16(s0, vmovl_u8(d0.val[1]), gc);
        s0 = vmlaq_u16(s0, vmovl_u8(d0.val[2]), bc);
        vst1_u8(&gray[i + 0], vshrn_n_u16(s0, 8));

        uint8x8x3_t d1 = vld3_u8(p + 24);
        uint16x8_t s1 = vmulq_u16(vmovl_u8(d1.val[0]), rc);
        s1 = vmlaq_u16(s1, vmovl_u8(d1.val[1]), gc);
        s1 = vmlaq_u16(s1, vmovl_u8(d1.val[2]), bc);
        vst1_u8(&gray[i + 8], vshrn_n_u16(s1, 8));

        uint8x8x3_t d2 = vld3_u8(p + 48);
        uint16x8_t s2 = vmulq_u16(vmovl_u8(d2.val[0]), rc);
        s2 = vmlaq_u16(s2, vmovl_u8(d2.val[1]), gc);
```

```
        s2 = vmlaq_u16(s2, vmovl_u8(d2.val[2]), bc);
        vst1_u8(&gray[i + 16], vshrn_n_u16(s2, 8));

        uint8x8x3_t d3 = vld3_u8(p + 72);
        uint16x8_t s3 = vmulq_u16(vmovl_u8(d3.val[0]), rc);
        s3 = vmlaq_u16(s3, vmovl_u8(d3.val[1]), gc);
        s3 = vmlaq_u16(s3, vmovl_u8(d3.val[2]), bc);
        vst1_u8(&gray[i + 24], vshrn_n_u16(s3, 8));

        uint8x8x3_t d4 = vld3_u8(p + 96);
        uint16x8_t s4 = vmulq_u16(vmovl_u8(d4.val[0]), rc);
        s4 = vmlaq_u16(s4, vmovl_u8(d4.val[1]), gc);
        s4 = vmlaq_u16(s4, vmovl_u8(d4.val[2]), bc);
        vst1_u8(&gray[i + 32], vshrn_n_u16(s4, 8));

        uint8x8x3_t d5 = vld3_u8(p + 120);
        uint16x8_t s5 = vmulq_u16(vmovl_u8(d5.val[0]), rc);
        s5 = vmlaq_u16(s5, vmovl_u8(d5.val[1]), gc);
        s5 = vmlaq_u16(s5, vmovl_u8(d5.val[2]), bc);
        vst1_u8(&gray[i + 40], vshrn_n_u16(s5, 8));

        uint8x8x3_t d6 = vld3_u8(p + 144);
        uint16x8_t s6 = vmulq_u16(vmovl_u8(d6.val[0]), rc);
        s6 = vmlaq_u16(s6, vmovl_u8(d6.val[1]), gc);
        s6 = vmlaq_u16(s6, vmovl_u8(d6.val[2]), bc);
        vst1_u8(&gray[i + 48], vshrn_n_u16(s6, 8));

        uint8x8x3_t d7 = vld3_u8(p + 168);
        uint16x8_t s7 = vmulq_u16(vmovl_u8(d7.val[0]), rc);
        s7 = vmlaq_u16(s7, vmovl_u8(d7.val[1]), gc);
        s7 = vmlaq_u16(s7, vmovl_u8(d7.val[2]), bc);
        vst1_u8(&gray[i + 56], vshrn_n_u16(s7, 8));
    }

    for (; i < n; i++) {
        gray[i] = (77 * rgb[i * 3] + 150 * rgb[i * 3 + 1] + 29 * rgb[i * 3 + 2]) >> 8;
    }
}
```

## 5.十六路循环展开

```
#include <arm_neon.h>
#include <stdint.h>

void rgb2gray_neon_x16(const uint8_t *rgb, uint8_t *gray, int n)
{
    uint16x8_t rc = vdupq_n_u16(77);
    uint16x8_t gc = vdupq_n_u16(150);
    uint16x8_t bc = vdupq_n_u16(29);
    int i;

    for (i = 0; i <= n - 128; i += 128) {
        const uint8_t *p = &rgb[i * 3];

        #define DO_GROUP(idx)                                          \
            do {                                                        \
                uint8x8x3_t d = vld3_u8(p + 24 * idx);                  \
                uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);      \
                s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);              \
                s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);              \
                vst1_u8(&gray[i + 8 * idx], vshrn_n_u16(s, 8));        \
            } while(0)

        DO_GROUP( 0); DO_GROUP( 1); DO_GROUP( 2); DO_GROUP( 3);
        DO_GROUP( 4); DO_GROUP( 5); DO_GROUP( 6); DO_GROUP( 7);
        DO_GROUP( 8); DO_GROUP( 9); DO_GROUP(10); DO_GROUP(11);
        DO_GROUP(12); DO_GROUP(13); DO_GROUP(14); DO_GROUP(15);
```

```
        #undef DO_GROUP
    }

    for (; i < n; i++)
        gray[i] = (77 * rgb[i*3] + 150 * rgb[i*3+1] + 29 * rgb[i*3+2]) >> 8;
}



#include <arm_neon.h>
#include <stdint.h>

void rgb2gray_neon_x16(const uint8_t *rgb, uint8_t *gray, int n)
{
    uint16x8_t rc = vdupq_n_u16(77);
    uint16x8_t gc = vdupq_n_u16(150);
    uint16x8_t bc = vdupq_n_u16(29);
    int i;

    for (i = 0; i <= n - 128; i += 128) {
        const uint8_t *p = &rgb[i * 3];

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 0);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 0], vshrn_n_u16(s, 8));
        }

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 1);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 1], vshrn_n_u16(s, 8));
        }

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 2);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 2], vshrn_n_u16(s, 8));
        }

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 3);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 3], vshrn_n_u16(s, 8));
        }

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 4);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 4], vshrn_n_u16(s, 8));
        }

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 5);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
```

```
    vst1_u8(&gray[i + 8 * 5], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 6);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 6], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 7);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 7], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 8);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 8], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 9);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 9], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 10);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 10], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 11);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 11], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 12);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 12], vshrn_n_u16(s, 8));
}

{
    uint8x8x3_t d = vld3_u8(p + 24 * 13);
    uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
    s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
    s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
    vst1_u8(&gray[i + 8 * 13], vshrn_n_u16(s, 8));
}

{
```

```
            uint8x8x3_t d = vld3_u8(p + 24 * 14);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 14], vshrn_n_u16(s, 8));
        }

        {
            uint8x8x3_t d = vld3_u8(p + 24 * 15);
            uint16x8_t s = vmulq_u16(vmovl_u8(d.val[0]), rc);
            s = vmlaq_u16(s, vmovl_u8(d.val[1]), gc);
            s = vmlaq_u16(s, vmovl_u8(d.val[2]), bc);
            vst1_u8(&gray[i + 8 * 15], vshrn_n_u16(s, 8));
        }
    }

    for (; i < n; i++) {
        gray[i] = (77 * rgb[i * 3] + 150 * rgb[i * 3 + 1] + 29 * rgb[i * 3 + 2]) >> 8;
    }
}
```